

```
// File: Sources/xlsOne/XlsOneApp.swift
```

```
import AppKit
import SwiftUI
import xlsOneCore
import xlsOneUI
```

```
@main
struct XlsOneApp: App {
    @NSApplicationDelegateAdaptor(WorkspaceAppDelegate.self) private var appDelegate

    init() {
        NSWindow.allowsAutomaticWindowTabbing = false
        LocaleManager.shared.applyToFoundation()
    }

    var body: some Scene {
        XlsOneWorkspaceScene()
    }
}
```

```
// File: Sources/xlsOneUI/WorkspaceScene.swift
```

```
import AppKit
import SwiftUI
import UniformTypeIdentifiers
import xlsOneCore
import xlsOneLicense
```

```
public struct XlsOneWorkspaceScene: Scene {
    @StateObject private var viewModel = AppViewModel()
    @StateObject private var licenseManager = LicenseManager.shared
    @ObservedObject private var localeManager = LocaleManager.shared

    public init() {}

    public var body: some Scene {
        WindowGroup {
            ContentView()
                .environmentObject(viewModel)
                .environmentObject(licenseManager)
                .environment(\.locale, localeManager.swiftUILocale)
                .frame(minWidth: 800, minHeight: 600)
                .onAppear {
                    // Menu localization is handled in applicationDidFinishLaunching
                }
                .background(WorkspaceWindowTabDisabler())
        }
        .commands {
            WorkspaceCommands(viewModel: viewModel, licenseManager: licenseManager, localeManager: localeManager)
        }
    }
}

@objc private final class AppAboutHelpHandler: NSObject {
    static let shared = AppAboutHelpHandler()

    @objc func showCustomAbout() {
        DispatchQueue.main.async {
            WorkspaceAppPresentation.showAboutPanel()
        }
    }
}
```

```

    }
}

private struct WorkspaceCommands: Commands {
    @ObservedObject var viewModel: AppViewModel
    @ObservedObject var licenseManager: LicenseManager
    @ObservedObject var localeManager: LocaleManager

    var body: some Commands {
        WorkspaceFileCommands(viewModel: viewModel, localeManager: localeManager)
        WorkspaceEditCommands(viewModel: viewModel, localeManager: localeManager)
        WorkspaceAdjustmentMemoryCommands(viewModel: viewModel, localeManager: localeManager)
        WorkspaceViewCommands(localeManager: localeManager)
        WorkspaceWindowCommands(localeManager: localeManager)
        WorkspaceLicenseCommands(licenseManager: licenseManager, localeManager: localeManager)
        WorkspaceLanguageCommands(localeManager: localeManager)
        WorkspaceHelpCommands(localeManager: localeManager)
    }
}

private struct WorkspaceFileCommands: Commands {
    @ObservedObject var viewModel: AppViewModel
    @ObservedObject var localeManager: LocaleManager

    var body: some Commands {
        CommandGroup(replacing: .appInfo) {
            Button(LocaleManager.loc("关于 表表归一")) {
                WorkspaceAppPresentation.showAboutPanel()
            }
        }

        CommandGroup(replacing: .newItem) {
            Button(LocaleManager.loc("新建批次")) {
                viewModel.closeAllFiles()
            }
            .keyboardShortcut("n", modifiers: .command)
        }

        CommandGroup(after: .newItem) {
            Divider()

            Button(LocaleManager.loc("导入文件...")) {
                viewModel.showOpenFileDialog()
            }
            .keyboardShortcut("o", modifiers: .command)

            Button(LocaleManager.loc("追加文件...")) {
                viewModel.showAddFileDialog()
            }
            .keyboardShortcut("o", modifiers: [.command, .shift])
            .disabled(!viewModel.toolbarPresentation.appendEnabled)
        }

        CommandGroup(replacing: .saveItem) {
            Button(LocaleManager.loc("导出 XLSX...")) {
                viewModel.exportResult()
            }
            .keyboardShortcut("s", modifiers: .command)
            .disabled(!viewModel.canExport)
        }
    }
}

```

```

CommandGroup(after: .saveItem) {
    Divider()

    Button(LocaleManager.loc("刷新")) {
        viewModel.reloadFiles()
    }
    .keyboardShortcut("r", modifiers: .command)
    .disabled(viewModel.selectedFilePaths.isEmpty)
}
}

private struct WorkspaceEditCommands: Commands {
    @ObservedObject var viewModel: AppViewModel
    @ObservedObject var localeManager: LocaleManager

    var body: some Commands {
        CommandGroup(replacing: .undoRedo) {
            Button(LocaleManager.loc("撤销上一步")) {
                viewModel.undoLastOverride()
            }
            .keyboardShortcut("z", modifiers: .command)
            .disabled(!viewModel.canUndoOverride)

            Divider()

            Button(LocaleManager.loc("清除本批次调整")) {
                viewModel.clearOverrides()
            }
            .disabled(viewModel.correctionCount == 0)
        }

        CommandGroup(replacing: .pasteboard) {}
    }
}

private struct WorkspaceAdjustmentMemoryCommands: Commands {
    @ObservedObject var viewModel: AppViewModel
    @ObservedObject var localeManager: LocaleManager

    var body: some Commands {
        CommandMenu(LocaleManager.loc("调整记忆")) {
            Button(LocaleManager.loc("查看当前调整记忆")) {
                viewModel.showSchemaManagerWindow()
            }
            .keyboardShortcut(",", modifiers: .command)
            .disabled(!viewModel.canManageAdjustmentMemory)

            Button(LocaleManager.loc("保存当前调整记忆")) {
                viewModel.saveCurrentAdjustmentMemory()
            }
            .disabled(!viewModel.canManageAdjustmentMemory)
        }
    }
}

private struct WorkspaceViewCommands: Commands {
    @ObservedObject var localeManager: LocaleManager

```

```

var body: some Commands {
    CommandGroup(replacing: .toolbar) {}
}
}

private struct WorkspaceWindowCommands: Commands {
    @ObservedObject var localeManager: LocaleManager

    var body: some Commands {
        CommandGroup(replacing: .windowArrangement) {
            Button(LocaleManager.loc("最小化")) {
                NSApp.keyWindow?.miniaturize(nil)
            }
            .keyboardShortcut("m", modifiers: .command)

            Button(LocaleManager.loc("缩放")) {
                NSApp.keyWindow?.zoom(nil)
            }

            Divider()

            Button(LocaleManager.loc("前置全部窗口")) {
                NSApp.arrangeInFront(nil)
            }
        }
    }
}

private struct WorkspaceLicenseCommands: Commands {
    @ObservedObject var licenseManager: LicenseManager
    @ObservedObject var localeManager: LocaleManager

    var body: some Commands {
        CommandMenu(LocaleManager.loc("许可")) {
            Button(LocaleManager.loc("激活/导入许可证...")) {
                licenseManager.showActivationSheet = true
            }
            .disabled(licenseManager.licenseState == .activated)
        }
    }
}

private struct WorkspaceLanguageCommands: Commands {
    @ObservedObject var localeManager: LocaleManager

    var body: some Commands {
        CommandMenu(LocaleManager.loc("language_menu")) {
            ForEach(LocaleManager.AppLanguage.allCases) { language in
                Button {
                    // Save-only: UI and menus stay unchanged until next launch.
                    UserDefaults.standard.set(language.rawValue, forKey: "AppLanguage")
                    localeManager.applyToFoundation(for: language)
                    showRestartToast(for: language)
                } label: {
                    HStack {
                        Text(language.displayName)
                        if localeManager.currentLanguage == language {
                            Image(systemName: "checkmark")
                        }
                    }
                }
            }
        }
    }
}

```

```

        }
    }
}

private struct WorkspaceHelpCommands: Commands {
    @ObservedObject var localeManager: LocaleManager

    var body: some Commands {
        CommandGroup(replacing: .help) {
            Button(LocaleManager.loc("检查更新")) {
                WorkspaceAppPresentation.showAppStoreUpdatePanel()
            }

            Divider()

            Button(LocaleManager.loc("使用帮助")) {
                WorkspaceAppPresentation.showHelpPanel()
            }

            Divider()

            Button(LocaleManager.loc("关于 表表归一")) {
                WorkspaceAppPresentation.showAboutPanel()
            }
        }
    }
}

private struct WorkspaceWindowTabDisabler: NSViewRepresentable {
    func makeNSView(context: Context) -> WorkspaceWindowTabDisablingView {
        WorkspaceWindowTabDisablingView()
    }

    func updateNSView(_ nsView: WorkspaceWindowTabDisablingView, context: Context) {
        nsView.disableWindowTabs()
    }
}

private final class WorkspaceWindowTabDisablingView: NSView {
    override func viewDidMoveToWindow() {
        super.viewDidMoveToWindow()
        disableWindowTabs()
    }

    func disableWindowTabs() {
        DispatchQueue.main.async { [weak self] in
            guard let window = self?.window else { return }
            WorkspaceWindowTabController.disableTabs(for: window)
        }
    }
}

private enum WorkspaceWindowTabController {
    @MainActor
    static func disableAutomaticTabbing() {
        NSWindow.allowsAutomaticWindowTabbing = false
    }
}

```

```

@MainActor
static func disableTabs(for window: NSWindow) {
    disableAutomaticTabbing()
    window.title = WorkspaceAppPresentation.localizedCaption
    window.tabbingMode = .disallowed
    window.tabbingIdentifier = ""
    if window.tabbedWindows != nil {
        window.toggleTabBar(nil)
    }
}
}

enum WorkspaceDialogPresenter {
    @MainActor
    private static var ownerWindow: NSWindow? {
        NSApp.keyWindow ?? NSApp.mainWindow ?? NSApp.windows.first { $0.isVisible }
    }

    @MainActor
    private static func center(_ dialogWindow: NSWindow, over owner: NSWindow?) {
        guard let owner else {
            dialogWindow.center()
            return
        }

        let ownerFrame = owner.frame
        let dialogFrame = dialogWindow.frame
        let origin = NSPoint(
            x: ownerFrame.midX - dialogFrame.width / 2,
            y: ownerFrame.midY - dialogFrame.height / 2
        )
        dialogWindow setFrameOrigin(origin)
    }

    @MainActor
    @discardableResult
    static func runModal(_ panel: NSSavePanel) -> NSApplication.ModalResponse {
        let owner = ownerWindow
        panel.contentView?.layoutSubtreeIfNeeded()
        center(panel, over: owner)
        return panel.runModal()
    }

    @MainActor
    @discardableResult
    static func runAlert(_ alert: NSAlert) -> NSApplication.ModalResponse {
        let owner = ownerWindow
        alert.layout()
        center(alert.window, over: owner)
        DispatchQueue.main.async {
            center(alert.window, over: owner)
        }
        return alert.runModal()
    }

    @MainActor
    static func runAlert(title: String, message: String, style: NSAlert.Style = .warning) {
        let alert = NSAlert()
        alert.alertStyle = style
        alert.messageText = title
    }
}

```

```

        alert.informativeText = message
        alert.addButton(withTitle: LocaleManager.Loc("确定"))
        runAlert(alert)
    }
}

@MainActor
func showRestartToast(for language: LocaleManager.AppLanguage) {
    NotificationCenter.default.post(name: .showRestartToast, object: language)
}

extension Notification.Name {
    static let showRestartToast = Notification.Name("showRestartToast")
}

public final class WorkspaceAppDelegate: NSObject, NSApplicationDelegate {
    public func applicationDidFinishLaunching(_ notification: Notification) {
        WorkspaceWindowTabController.disableAutomaticTabbing()
    }
}

enum WorkspaceAppPresentation {
    static var localizedCaption: String {
        isPreferredLanguageChinese ? "xlsOne 表表归一" : "xlsOne"
    }

    static var displayName: String {
        localizedCaption
    }

    static var bundleDisplayName: String {
        Bundle.main.object(forKey: "CFBundleDisplayName") as? String ??
        Bundle.main.object(forKey: "CFBundleName") as? String ??
        "xlsOne"
    }

    static var versionLabel: String {
        let version = Bundle.main.object(forKey: "CFBundleShortVersionString") as? String
        let build = Bundle.main.object(forKey: "CFBundleVersion") as? String

        switch (version, build) {
        case let (.some(version), .some(build)) where !version.isEmpty && !build.isEmpty && version != build:
            return "版本 \(version) (\(build))"
        case let (.some(version), _):
            return "版本 \(version)"
        case let (_, .some(build)):
            return "构建 \(build)"
        default:
            return "版本信息不可用"
        }
    }

    private static var marketingVersion: String {
        Bundle.main.object(forKey: "CFBundleShortVersionString") as? String ?? "1.0"
    }

    private static var isPreferredLanguageChinese: Bool {
        LocaleManager.shared.currentLanguage.isChineseLike()
    }
}

```

```
@MainActor
```

```
static func showAboutPanel() {
    let alert = NSAlert()
    let isChinese = isPreferredLanguageChinese
    alert.messageText = isChinese ? "关于 表表归一" : "About xlsOne"
    alert.informativeText = isChinese ? ""
        表表归一 V\(\marketingVersion)
```

多张同格式 Excel 报表一键汇总

把多张格式一致的 Excel 表合成一份汇总表。金额、数量等能相加的数会自动合计；名称、编号等不该相加的信息，会保留各文件里最常见的共同前缀。

作者：王臻

技术：Swift / SwiftUI

邮箱：831261@qq.com

© 2026 王臻. 保留所有权利.

"" : ""

xlsOne V\(\marketingVersion)

One-click summary for multiple identically-formatted Excel reports.

Merge multiple Excel sheets with identical layouts into a single summary sheet. Amounts, quantities, and other summable values are automatically totaled. Names, codes, and other non-summable information retain the most common values across all files.

Author: Wang Zhen

Tech: Swift / SwiftUI

Email: 831261@qq.com

© 2026 Wang Zhen. All rights reserved.

""

```
alert.addButton(withTitle: LocaleManager.loc("确定"))
WorkspaceDialogPresenter.runAlert(alert)
NSApp.activate(ignoringOtherApps: true)
}
```

```
@MainActor
```

```
static func showHelpPanel() {
    let alert = NSAlert()
    let isChinese = isPreferredLanguageChinese
    alert.messageText = isChinese ? "使用帮助" : "Help"
    alert.informativeText = isChinese ? ""
        表表归一 · 多张同格式 Excel 报表一键汇总
```

1. 导入文件

拖拽 .xlsx 或 .xls 文件到窗口，或点击 [文件] → [导入文件]

2. 切换工作表

点击顶部的 Sheet 标签可切换要查看的报表页

3. 查看汇总

- 金额、数量等能相加的数自动合计
- 名称、编号等信息保留最常见的共同前缀
- 结构不一致的工作表会跳过，并提示原因

4. 穿透查阅

点击单元格可查看各文件原始值

5. 导出结果

点击 [导出 XLSX] 保存汇总结果

6. 单元格修正

在右侧面板可将单元格手动指定为标签或求和

快捷键:

Command+O 导入文件
 Command+Shift+O 追加文件
 Command+S 导出
 Command+R 刷新
 Command+N 清空
 Command+Z 撤销修正

"" : ""

xlsOne · One-click Excel Report Merger

1. Import Files

Drag .xlsx or .xls files into the window, or click [File] → [Import Files]

2. Switch Sheets

Click the Sheet tabs at the top to switch between report pages

3. View Summary

- Amounts, quantities, and summable numbers are automatically totaled
- Names, codes retain the most common values
- Sheets with inconsistent structure are skipped with a reason

4. Source Drilldown

Click any cell to view its raw values from each source file

5. Export Results

Click [Export XLSX] to save the summary

6. Cell Correction

Manually override cell type to Label or Sum in the right panel

Shortcuts:

Cmd+O Import Files
 Cmd+Shift+O Add Files
 Cmd+S Export
 Cmd+R Refresh
 Cmd+N Clear
 Cmd+Z Undo Correction

""

```
alert.addButton(withTitle: LocaleManager.loc("确定"))
WorkspaceDialogPresenter.runAlert(alert)
NSApp.activate(ignoringOtherApps: true)
}
```

@MainActor

```
static func showAppStoreUpdatePanel() {
    WorkspaceDialogPresenter.runAlert(
        title: LocaleManager.loc("检查更新"),
        message: LocaleManager.loc("App Store 版本请通过 Mac App Store 获取更新。"),
        style: .informational
    )
    NSApp.activate(ignoringOtherApps: true)
}
}
```

/// 应用状态管理

@MainActor

```
class AppViewModel: ObservableObject {
```

```

@Published var files: [ExcelFile] = []
@Published var selectedFilePaths: [String] = []
@Published var availableSheets: [String] = []
@Published var mergedResult: MergedResult?
@Published var validationReport: WorkbookValidationReport?
@Published var workspacePhase: WorkspacePhase = .idle
@Published var selectedCell: CellPosition?
@Published var anomalyQueue: [CellAnomalyItem] = []
@Published var allAnomalies: [CellAnomalyItem] = []
@Published var isLoading = false
@Published var errorMessage: String?
@Published var showError = false
@Published var selectedSheetSelection: WorkspaceSheetSelection?

```

```

private let parser = ExcelParser()
private let validator = WorkbookValidator()
private let smartMerger = SmartMerger()
private var mergedResultsBySheet: [String: MergedResult] = [:]

```

/// 显示打开文件对话框

```

func showOpenFileDialog() {
    let panel = NSOpenPanel()
    panel.allowsMultipleSelection = true
    panel.canChooseDirectories = false
    panel.canChooseFiles = true
    panel.allowedContentTypes = [
        UTType(filenameExtension: "xlsx")!,
        UTType(filenameExtension: "xls")!
    ]

    if WorkspaceDialogPresenter.runModal(panel) == .OK {
        let paths = panel.urls.map { $0.path }
        loadFiles(at: paths, append: !selectedFilePaths.isEmpty)
    }
}

```

/// 显示添加文件对话框

```

func showAddFileDialog() {
    let panel = NSOpenPanel()
    panel.allowsMultipleSelection = true
    panel.canChooseDirectories = false
    panel.canChooseFiles = true
    panel.allowedContentTypes = [
        UTType(filenameExtension: "xlsx")!,
        UTType(filenameExtension: "xls")!
    ]

    if WorkspaceDialogPresenter.runModal(panel) == .OK {
        let paths = panel.urls.map { $0.path }
        loadFiles(at: paths, append: true)
    }
}

```

/// 加载文件

```

func loadFiles(at paths: [String], append: Bool) {
    selectedFilePaths = append
        ? Self.deduplicatedPaths(selectedFilePaths + paths)
        : Self.deduplicatedPaths(paths)
    revalidateSelection()
}

```

```

/// 关闭所有文件
func closeAllFiles() {
    files.removeAll()
    selectedFilePaths.removeAll()
    availableSheets.removeAll()
    mergedResult = nil
    validationReport = nil
    workspacePhase = .idle
    mergedResultsBySheet.removeAll()
    selectedCell = nil
    anomalyQueue.removeAll()
    allAnomalies.removeAll()
    matchedSchema = nil
    workspaceRuleState = .none
    userOverrides.removeAll()
    forgottenOverrideKeys.removeAll()
    overrideHistory.removeAll()
    workspaceBaseOverrides.removeAll()
    workspaceBaseSchemaID = nil
    workspaceActiveSchemaID = nil
    selectedSheetSelection = nil
}

/// 重新加载文件
func reloadFiles() {
    revalidateSelection()
}

/// 重置
func reset() {
    closeAllFiles()
}

/// 切换工作表
func switchToSheet(_ sheetName: String) {
    selectedSheetSelection = .mergeable(sheetName)
    syncSelectedSheetPresentation()
    refreshSelectionAndAnomalies()
}

/// 切换到跳过的工作表
func switchToSkippedSheet(_ sheetName: String) {
    selectedSheetSelection = .skipped(sheetName)
    syncSelectedSheetPresentation()
    refreshSelectionAndAnomalies()
}

/// 当前用户自定义的单元格覆盖
@Published var userOverrides: [CellTypeOverride] = []

/// 覆盖操作历史（用于撤销）
private var overrideHistory: [OverrideOperation] = []

/// 本批次明确要求移除的已记住调整
private var forgottenOverrideKeys: Set<StoredOverrideKey> = []

/// 当前工作区在本批次开始前的已记住调整快照
private var workspaceBaseOverrides: [CellTypeOverride] = []
private var workspaceBaseSchemaID: UUID?

```

```

private var workspaceActiveSchemaID: UUID?

/// 是否显示规则管理器
@Published var showSchemaManager = false

/// 当前匹配的规则
@Published var matchedSchema: MergeSchema?

/// 当前工作区的规则应用状态
@Published var workspaceRuleState: WorkspaceRuleState = .none

var canManageAdjustmentMemory: Bool {
    workspacePhase == .ready && !files.isEmpty && !availableSheets.isEmpty
}

var currentAdjustmentMemory: MergeSchema? {
    matchedSchema
}

var canExport: Bool {
    workspacePhase == .ready &&
    validationReport?.readiness == .ready &&
    !files.isEmpty &&
    !mergedResultsBySheet.isEmpty
}

var correctionCount: Int {
    userOverrides.count + forgottenOverrideKeys.count
}

var canUndoOverride: Bool {
    !overrideHistory.isEmpty
}

var manualOverridePositionsForCurrentSheet: Set<CellPosition> {
    guard let currentSheet else { return [] }
    return Set(
        userOverrides
        .filter { $0.sheetName == currentSheet }
        .map { CellPosition(row: $0.rowIndex, col: $0.colIndex) }
    )
}

var participatingFileCount: Int {
    validationReport?.includedFiles.count ?? 0
}

var blockedFileCount: Int {
    validationReport?.blockedFiles.count ?? 0
}

var warningFileCount: Int {
    validationReport?.warningFiles.count ?? 0
}

var sheetOverviewItems: [SheetOverviewItem] {
    guard let validationReport else { return [] }
    return WorkspaceDiagnostics.buildSheetOverview(
        report: validationReport,
        anomalyItems: allAnomalies
    )
}

```

```

    )
}

var toolbarPresentation: ToolbarPresentation {
    WorkspaceToolbar.buildPresentation(
        selectedFileCount: selectedFilePaths.count,
        canExport: canExport
    )
}

var selectedCellReference: String? {
    guard let selectedCell else { return nil }
    return WorkspaceDiagnostics.cellReference(row: selectedCell.row, col: selectedCell.col)
}

var currentSheet: String? {
    guard case .mergeable(let sheetName)? = selectedSheetSelection else { return nil }
    return sheetName
}

var selectedSheetName: String? {
    selectedSheetSelection?.sheetName
}

var selectedSheetStructureStatus: String {
    switch selectedSheetSelection {
    case .mergeable:
        return "可合并"
    case .skipped:
        return "已跳过"
    case .none:
        return validationReport?.readiness == .ready ? "可合并" : "-"
    }
}

var selectedSkippedSheetConsensus: SkippedSheetConsensus? {
    guard case .skipped(let sheetName)? = selectedSheetSelection,
        let validationReport else {
        return nil
    }
    return WorkspaceDiagnostics.buildSkippedSheetConsensus(
        report: validationReport,
        sheetName: sheetName
    )
}

var selectedMergedCell: MergedCell? {
    guard let selectedCell,
        let result = mergedResult,
        selectedCell.row < result.rows.count,
        selectedCell.col < result.rows[selectedCell.row].count else {
        return nil
    }
    return result.rows[selectedCell.row][selectedCell.col]
}

func correctionState(for position: CellPosition, cell: MergedCell? = nil) -> CellCorrectionState {
    if manualOverridePositionsForCurrentSheet.contains(position) {
        return .manual
    }
}

```

```

if let cell {
    return cell.isOverridden ? .rule : .none
}

guard let result = mergedResult,
    position.row < result.rows.count,
    position.col < result.rows[position.row].count else {
    return .none
}

return result.rows[position.row][position.col].isOverridden ? .rule : .none
}

var canRestoreSelectedCellAutomatic: Bool {
    guard let currentSheet,
        let selectedCell else {
        return false
    }

    return effectiveWorkspaceOverrides().contains {
        $0.sheetName == currentSheet &&
        $0.rowIndex == selectedCell.row &&
        $0.colIndex == selectedCell.col
    }
}

/// 更新合并结果
private func revalidateSelection() {
    guard !selectedFilePaths.isEmpty else {
        closeAllFiles()
        return
    }

    isLoading = true
    workspacePhase = .validating

    Task {
        let batch = await parser.parseFilesWithDiagnostics(at: selectedFilePaths)
        let outcome = validator.validate(files: batch.files, parseFailures: batch.failures)

        await MainActor.run {
            validationReport = outcome.report
            files = outcome.mergeableFiles
            availableSheets = outcome.report.commonSheetNames
            isLoading = false
        }

        guard outcome.report.readiness == .ready, !outcome.mergeableFiles.isEmpty else {
            await MainActor.run {
                workspacePhase = .blocked
                selectedSheetSelection = nil
                mergedResult = nil
                mergedResultsBySheet.removeAll()
                selectedCell = nil
                anomalyQueue.removeAll()
                allAnomalies.removeAll()
                matchedSchema = nil
                workspaceRuleState = .none
            }
        }
    }
}

```

```

        return
    }

    await MainActor.run {
        workspacePhase = .ready
        selectedSheetSelection = normalizedSelectedSheetSelection(report: outcome.report)
    }
    await refreshWorkspaceResults()
}

private func refreshWorkspaceResults() async {
    guard workspacePhase == .ready, !files.isEmpty else { return }

    let ruleMatchResult = await smartMerger.prepareWorkspaceSchema(
        files: files,
        sheetNames: availableSheets
    )
    let workspaceSchema: MergeSchema?
    switch ruleMatchResult {
    case .exact(let schema):
        workspaceSchema = schema
    case .ambiguous, .similar, .none:
        workspaceSchema = nil
    }

    syncWorkspaceMemoryBase(with: workspaceSchema)
    let effectiveOverrides = effectiveWorkspaceOverrides()
    var resultsBySheet: [String: MergedResult] = [:]
    for sheetName in availableSheets {
        let result = await smartMerger.merge(
            files: files,
            sheetName: sheetName,
            applying: nil
        )
        let finalResult = smartMerger.applyOverrides(to: result, overrides: effectiveOverrides)
        resultsBySheet[sheetName] = finalResult
    }

    await MainActor.run {
        mergedResultsBySheet = resultsBySheet
        syncSelectedSheetPresentation()
        matchedSchema = workspaceSchema
        workspaceRuleState = Self.ruleState(from: ruleMatchResult)
        refreshSelectionAndAnomalies()
    }
}

/// 应用单元格类型覆盖
func applyCellOverride(row: Int, col: Int, type: CellOverrideType) {
    guard let currentSheet else { return }
    recordOverrideSnapshot()
    userOverrides.removeAll {
        $0.sheetName == currentSheet &&
        $0.rowIndex == row &&
        $0.colIndex == col
    }
    forgottenOverrideKeys.remove(
        StoredOverrideKey(sheetName: currentSheet, rowIndex: row, colIndex: col)
    )
}

```

```

let override = CellTypeOverride(
    sheetName: currentSheet,
    rowIndex: row,
    colIndex: col,
    cellType: type,
    userNote: nil
)
userOverrides.append(override)

Task {
    await persistAdjustmentMemoryAndRefresh()
}

}

/// 批量应用类型覆盖
func applyBulkOverride(positions: [CellPosition], type: CellOverrideType) {
    guard let currentSheet, !positions.isEmpty else { return }

    recordOverrideSnapshot()
    for pos in positions {
        userOverrides.removeAll {
            $0.sheetName == currentSheet &&
            $0.rowIndex == pos.row &&
            $0.colIndex == pos.col
        }
        forgottenOverrideKeys.remove(
            StoredOverrideKey(sheetName: currentSheet, rowIndex: pos.row, colIndex: pos.col)
        )
    }

    for pos in positions {
        let override = CellTypeOverride(
            sheetName: currentSheet,
            rowIndex: pos.row,
            colIndex: pos.col,
            cellType: type,
            userNote: nil
        )
        userOverrides.append(override)
    }

    Task {
        await persistAdjustmentMemoryAndRefresh()
    }
}

/// 清除所有用户覆盖
func clearOverrides() {
    guard correctionCount > 0 else { return }
    recordOverrideSnapshot()
    userOverrides.removeAll()
    forgottenOverrideKeys.removeAll()
    Task {
        await persistAdjustmentMemoryAndRefresh()
    }
}

/// 撤销上一步覆盖操作
func undoLastOverride() {

```



```

guard let lastOperation = overrideHistory.popLast() else { return }

switch lastOperation {
case .snapshot(let previousOverrides, let previousForgottenKeys):
    userOverrides = previousOverrides
    forgottenOverrideKeys = previousForgottenKeys
}

Task {
    await persistAdjustmentMemoryAndRefresh()
}

}

/// 显示规则管理器
func showSchemaManagerWindow() {
    showSchemaManager = true
}

func saveCurrentAdjustmentMemory() {
    guard canManageAdjustmentMemory else {
        WorkspaceDialogPresenter.runAlert(
            title: LocaleManager.loc("保存当前调整记忆"),
            message: LocaleManager.loc("当前没有可保存调整记忆的同构工作区。"),
            style: .informational
        )
        return
    }

    Task {
        await persistAdjustmentMemoryAndRefresh()
    }

}

func exportCurrentAdjustmentMemory() {
    guard let schema = currentAdjustmentMemory else { return }

    Task {
        do {
            let data = try await smartMerger.exportSchema(id: schema.id)
            let url = await MainActor.run {
                let panel = NSSavePanel()
                let fallbackName = schema.name.trimmingCharacters(in: .whitespacesAndNewLines).isEmpty
                    ? "adjustment-memory"
                    : schema.name
                panel.nameFieldStringValue = "\(fallbackName).json"
                panel.allowedContentTypes = [.json]
                return WorkspaceDialogPresenter.runModal(panel) == .OK ? panel.url : nil
            }
            if let url {
                try data.write(to: url)
            }
        } catch {
            await MainActor.run {
                errorMessage = "导出调整记忆失败: \(error.localizedDescription)"
                showError = true
            }
        }
    }

}
}

```

```

func showImportAdjustmentMemoryDialog() {
    guard canManageAdjustmentMemory else {
        WorkspaceDialogPresenter.runAlert(
            title: LocaleManager.loc("导入"),
            message: LocaleManager.loc("当前没有可绑定调整记忆的同构工作区。"),
            style: .informational
        )
        return
    }

    let panel = NSOpenPanel()
    panel.allowsMultipleSelection = false
    panel.canChooseDirectories = false
    panel.canChooseFiles = true
    panel.allowedContentTypes = [UTType.json]

    guard WorkspaceDialogPresenter.runModal(panel) == .OK, let url = panel.urls.first else { return }
    Task {
        await importCurrentAdjustmentMemory(from: url)
    }
}

func clearCurrentAdjustmentMemory() {
    guard let schemaID = workspaceActiveSchemaID ?? workspaceBaseSchemaID ?? matchedSchema?.id else { return }

    let alert = NSAlert()
    alert.alertStyle = .warning
    alert.messageText = LocaleManager.loc("清除当前调整记忆")
    alert.informativeText = LocaleManager.loc("删除当前调整记忆后，当前同构工作区将恢复自动判断。确定清除？")
    alert.addButton(withTitle: LocaleManager.loc("清除"))
    alert.addButton(withTitle: LocaleManager.loc("取消"))
    guard WorkspaceDialogPresenter.runAlert(alert) == .alertFirstButtonReturn else { return }

    Task {
        do {
            try await smartMerger.deleteSchema(id: schemaID)
            await MainActor.run {
                workspaceActiveSchemaID = nil
                workspaceBaseSchemaID = nil
                workspaceBaseOverrides.removeAll()
                userOverrides.removeAll()
                forgottenOverrideKeys.removeAll()
                overrideHistory.removeAll()
                matchedSchema = nil
                workspaceRuleState = .none
            }
            await refreshWorkspaceResults()
        } catch {
            await MainActor.run {
                errorMessage = "清除调整记忆失败: \(error.localizedDescription)"
                showError = true
            }
        }
    }
}

func selectCell(_ position: CellPosition?) {
    selectedCell = position
    refreshSelectionAndAnomalies()
}

```

```

func jumpToNextAnomaly() {
    jumpAcrossAnomalies(step: 1)
}

func jumpToPreviousAnomaly() {
    jumpAcrossAnomalies(step: -1)
}

private func jumpAcrossAnomalies(step: Int) {
    guard !allAnomalies.isEmpty else { return }

    let currentID = currentSheet.flatMap { sheetName in
        selectedCell.map { "\(sheetName)|\((WorkspaceDiagnostics.cellReference(row: $0.row, col: $0.col))" }
    }

    let currentIndex = currentID.flatMap { id in
        allAnomalies.firstIndex(where: { $0.id == id })
    } ?? (step > 0 ? -1 : 0)

    let nextIndex = (currentIndex + step + allAnomalies.count) % allAnomalies.count
    let target = allAnomalies[nextIndex]
    selectedSheetSelection = .mergeable(target.sheetName)
    syncSelectedSheetPresentation()
    selectedCell = target.position
    refreshSelectionAndAnomalies()
}

/// 导出当前结果
func exportResult() {
    guard canExport,
        let templatePath = validationReport?.templateFile?.filepath else { return }

    let panel = NSSavePanel()
    let exportName = ExportNaming.suggestedWorkbookName(
        from: validationReport?.files.map(\.filename) ??
            selectedFilePaths.map { URL(fileURLWithPath: $0).lastPathComponent }
    )
    panel.nameFieldStringValue = exportName
    panel.allowedContentTypes = [UTType(filenameExtension: "xlsx")!]

    if WorkspaceDialogPresenter.runModal(panel) == .OK, let url = panel.url {
        let results = availableSheets.compactMap { mergedResultsBySheet[$0] }
        let exporter = TemplateWorkbookExporter()
        Task {
            do {
                try exporter.exportWorkbook(
                    templatePath: templatePath,
                    results: results,
                    to: url.path
                )
            } catch {
                await MainActor.run {
                    errorMessage = "导出失败: \(error.localizedDescription)"
                    showError = true
                }
            }
        }
    }
}

```

```

private fun refreshSelectionAndAnomalies() {
    allAnomalies = availableSheets.flatMap { sheetName in
        mergedResultsBySheet[sheetName].map(WorkspaceDiagnostics.buildAnomalyQueue(for:)) ?? []
    }

    guard let currentSheet else {
        anomalyQueue = []
        selectedCell = nil
        return
    }

    anomalyQueue = mergedResultsBySheet[currentSheet].map(WorkspaceDiagnostics.buildAnomalyQueue(for:)) ?? []

    guard let result = mergedResult else {
        selectedCell = nil
        return
    }

    if let selectedCell,
        selectedCell.row < result.rows.count,
        selectedCell.col < result.rows[selectedCell.row].count {
        return
    }

    if let firstAnomaly = anomalyQueue.first {
        selectedCell = firstAnomaly.position
    } else if !result.rows.isEmpty, !result.rows[0].isEmpty {
        selectedCell = CellPosition(row: 0, col: 0)
    } else {
        selectedCell = nil
    }
}

private fun normalizedSelectedSheetSelection(report: WorkbookValidationReport?) -> WorkspaceSheetSelection? {
    if let selectedSheetSelection {
        switch selectedSheetSelection {
            case .mergeable(let sheetName) where availableSheets.contains(sheetName):
                return .mergeable(sheetName)
            case .skipped(let sheetName) where report?.skippedSheetNames.contains(sheetName) == true:
                return .skipped(sheetName)
            default:
                break
        }
    }

    if let firstMergeableSheet = availableSheets.first {
        return .mergeable(firstMergeableSheet)
    }

    if let firstSkippedSheet = report?.skippedSheetNames.first {
        return .skipped(firstSkippedSheet)
    }

    return nil
}

private fun syncSelectedSheetPresentation() {
    let normalizedSelection = normalizedSelectedSheetSelection(report: validationReport)
    if normalizedSelection != selectedSheetSelection {

```

```

        selectedSheetSelection = normalizedSelection
    }

    switch normalizedSelection {
    case .mergeable(let sheetName):
        mergedResult = mergedResultsBySheet[sheetName]
    case .skipped:
        mergedResult = nil
    case .none:
        mergedResult = nil
    }
}

func restoreAutomaticDecisionForSelectedCell() {
    guard let currentSheet,
          let selectedCell,
          canRestoreSelectedCellAutomatic else {
        return
    }

    recordOverrideSnapshot()
    userOverrides.removeAll {
        $0.sheetName == currentSheet &&
        $0.rowIndex == selectedCell.row &&
        $0.colIndex == selectedCell.col
    }

    let key = StoredOverrideKey(sheetName: currentSheet, rowIndex: selectedCell.row, colIndex: selectedCell.col)
    if workspaceBaseOverrides.contains(where: {
        StoredOverrideKey(override: $0) == key
    }) {
        forgottenOverrideKeys.insert(key)
    } else {
        forgottenOverrideKeys.remove(key)
    }

    Task {
        await persistAdjustmentMemoryAndRefresh()
    }
}

private static func deduplicatedPaths(_ paths: [String]) -> [String] {
    var seen: Set<String> = []
    var result: [String] = []
    for path in paths where !seen.contains(path) {
        seen.insert(path)
        result.append(path)
    }
    return result
}

private static func ruleState(from matchResult: SchemaMatchResult) -> WorkspaceRuleState {
    switch matchResult {
    case .exact(let schema):
        return .applied(name: schema.name, correctionCount: schema.cellOverrides.count)
    case .ambiguous(let candidates):
        return .ambiguous(count: candidates.count)
    case .similar(let candidates):
        return .similar(count: candidates.count)
    case .none:

```

```

        return .none
    }
}

private static func mergedRuleOverrides(
    existing: [CellTypeOverride],
    updates: [CellTypeOverride]
) -> [CellTypeOverride] {
    var merged = existing

    for update in updates {
        merged.removeAll {
            $0.sheetName == update.sheetName &&
            $0.rowIndex == update.rowIndex &&
            $0.colIndex == update.colIndex
        }
        merged.append(update)
    }

    return merged
}

private func recordOverrideSnapshot() {
    overrideHistory.append(
        .snapshot(
            previousOverrides: userOverrides,
            previousForgottenKeys: forgottenOverrideKeys
        )
    )
}

private func syncWorkspaceMemoryBase(with workspaceSchema: MergeSchema?) {
    guard userOverrides.isEmpty, forgottenOverrideKeys.isEmpty else { return }
    workspaceBaseOverrides = workspaceSchema?.cellOverrides ?? []
    workspaceBaseSchemaID = workspaceSchema?.id
    workspaceActiveSchemaID = workspaceSchema?.id
}

private func effectiveWorkspaceOverrides() -> [CellTypeOverride] {
    let rememberedOverrides = workspaceBaseOverrides.filter {
        !forgottenOverrideKeys.contains(StoredOverrideKey(override: $0))
    }
    return Self.mergedRuleOverrides(existing: rememberedOverrides, updates: userOverrides)
}

private func persistAdjustmentMemoryAndRefresh() async {
    do {
        try await persistAdjustmentMemory()
    } catch {
        errorMessage = "记住调整失败: \(error.localizedDescription)"
        showError = true
    }
    await refreshWorkspaceResults()
}

private func persistAdjustmentMemory() async throws {
    guard workspacePhase == .ready, !files.isEmpty, !availableSheets.isEmpty else { return }

    let fingerprint = FingerprintGenerator.generateWorkspaceLegacyFingerprint(
        from: files,

```

```

        sheetNames: availableSheets
    )
    let workbookFingerprint = FingerprintGenerator.generateWorkbook(
        from: files,
        sheetNames: availableSheets
    )
    let effectiveOverrides = effectiveWorkspaceOverrides()
    let schemaID = workspaceActiveSchemaID ?? workspaceBaseSchemaID

    if effectiveOverrides.isEmpty {
        if let schemaID {
            if workspaceBaseSchemaID == nil {
                try await smartMerger.deleteSchema(id: schemaID)
                workspaceActiveSchemaID = nil
            } else {
                _ = try await smartMerger.updateSchema(
                    id: schemaID,
                    overrides: [],
                    fingerprint: fingerprint,
                    workbookFingerprint: workbookFingerprint
                )
                workspaceActiveSchemaID = schemaID
            }
        }
        return
    }

    if let schemaID {
        _ = try await smartMerger.updateSchema(
            id: schemaID,
            overrides: effectiveOverrides,
            fingerprint: fingerprint,
            workbookFingerprint: workbookFingerprint
        )
        workspaceActiveSchemaID = schemaID
    } else {
        let schema = try await smartMerger.createSchema(
            name: suggestedAdjustmentMemoryName(),
            fingerprint: fingerprint,
            workbookFingerprint: workbookFingerprint,
            overrides: effectiveOverrides
        )
        workspaceActiveSchemaID = schema.id
    }
}

private func importCurrentAdjustmentMemory(from url: URL) async {
    do {
        let data = try Data(contentsOf: url)
        let schema = try await smartMerger.importSchema(
            data: data,
            forCurrentWorkspaceFiles: files,
            sheetNames: availableSheets
        )
        await MainActor.run {
            workspaceBaseOverrides = schema.cellOverrides
            workspaceBaseSchemaID = schema.id
            workspaceActiveSchemaID = schema.id
            userOverrides.removeAll()
            forgottenOverrideKeys.removeAll()
        }
    }
}

```

```

        overrideHistory.removeAll()
    }
    await refreshWorkspaceResults()
} catch {
    await MainActor.run {
        errorMessage = "导入调整记忆失败: \$(error.localizedDescription)"
        showError = true
    }
}
}

private func suggestedAdjustmentMemoryName() -> String {
    let exportName = ExportNaming.suggestedWorkbookName(
        from: validationReport?.files.map(\.filename) ??
            selectedFilePaths.map { URL(fileURLWithPath: $0).lastPathComponent }
    )

    if exportName.hasSuffix("_汇总") {
        return String(exportName.dropLast("_汇总".count)) + "_调整记忆"
    }
    if exportName.hasSuffix("汇总") {
        return String(exportName.dropLast("汇总".count)) + "调整记忆"
    }
    return "\$(exportName)_调整记忆"
}
}

// MARK: - 覆盖操作历史

private enum OverrideOperation {
    case snapshot(previousOverrides: [CellTypeOverride], previousForgottenKeys: Set<StoredOverrideKey>)
}

private struct StoredOverrideKey: Hashable {
    let sheetName: String
    let rowIndex: Int
    let colIndex: Int

    init(sheetName: String, rowIndex: Int, colIndex: Int) {
        self.sheetName = sheetName
        self.rowIndex = rowIndex
        self.colIndex = colIndex
    }

    init(override: CellTypeOverride) {
        self.sheetName = override.sheetName ?? ""
        self.rowIndex = override.rowIndex
        self.colIndex = override.colIndex
    }
}

// File: Sources/xlsOneUI/ContentView.swift
import SwiftUI
import xlsOneCore
import xlsOneLicense

struct ContentView: View {
    @EnvironmentObject var viewModel: AppViewModel
    @EnvironmentObject var licenseManager: LicenseManager
    @ObservedObject private var localeManager = LocaleManager.shared

```



```

@State private var isDropTargeted = false
@State private var showRestartToast = false
@State private var toastLanguage: LocaleManager.AppLanguage = .system
@State private var toastDismissTask: Task<Void, Never>?

var body: some View {
    VStack(spacing: 0) {
        toolbar
        Divider()

        switch viewModel.workspacePhase {
        case .idle:
            emptyView
        case .validating:
            loadingView
        case .blocked:
            blockedView
        case .ready:
            readyWorkspace
        }
    }
    .id(localeManager.currentLanguage.rawValue)
    .onChange(of: viewModel.showError) { isPresented in
        guard isPresented else { return }
        showErrorAlert()
    }
    .background {
        CenteredDialogWindow(
            isPresented: $viewModel.showSchemaManager,
            titleKey: "当前调整记忆",
            size: NSSize(width: 600, height: 400)
        ) {
            SchemaManagerView()
                .id(localeManager.currentLanguage.rawValue)
        }
        .frame(width: 0, height: 0)
    }
    .background {
        CenteredDialogWindow(
            isPresented: licenseActivationSheet,
            titleKey: "激活 表表归一",
            size: NSSize(width: 420, height: 520)
        ) {
            LicenseActivationView()
                .id(localeManager.currentLanguage.rawValue)
        }
        .frame(width: 0, height: 0)
    }
    .onDrop(of: [.fileURL], delegate: DropDelegateView(viewModel: viewModel, isTargeted: $isDropTargeted))
    .overlay {
        if showRestartToast {
            RestartToastView(language: toastLanguage)
                .transition(.opacity)
        }
    }
    .animation(.easeInOut(duration: 0.35), value: showRestartToast)
    .onReceive(NotificationCenter.default.publisher(for: .showRestartToast)) { notification in
        if let lang = notification.object as? LocaleManager.AppLanguage {
            toastLanguage = lang
        }
    }
}

```

```

        toastDismissTask?.cancel()
        withAnimation {
            showRestartToast = true
        }
        toastDismissTask = Task { @MainActor in
            try? await Task.sleep(nanoseconds: 4_500_000_000)
            guard !Task.isCancelled else { return }
            withAnimation {
                showRestartToast = false
            }
        }
    }
    .task {
        await licenseManager.verifyOnLaunch()
    }
}

private var licenseActivationSheet: Binding<Bool> {
    Binding(
        get: {
            if LicenseManager.isAppStoreDistribution { return false }
            let state = licenseManager.licenseState
            if state == .activated { return false }
            if licenseManager.showActivationSheet {
                return true
            }
            if case .trial = state { return false }
            return state == .unactivated || state == .expired
        },
        set: { newValue in
            if !newValue {
                licenseManager.showActivationSheet = false
            }
        }
    )
}

private func showErrorAlert() {
    WorkspaceDialogPresenter.runAlert(
        title: LocaleManager.loc("错误"),
        message: viewModel.errorMessage ?? LocaleManager.loc("未知错误"),
        style: .warning
    )
    viewModel.showError = false
}

private var toolbar: some View {
    let presentation = viewModel.toolbarPresentation
    let hasWorkspace = !viewModel.selectedFilePaths.isEmpty

    return HStack(spacing: 10) {
        if hasWorkspace {
            HStack(spacing: 2) {
                toolbarUtilityButton(label: LocLabel("追加", systemImage: "plus"), help: LocaleManager.loc("向当前批次追加文件")) {
                    viewModel.showAddFileDialog()
                }
                .disabled(!presentation.appendEnabled)

                toolbarStripDivider
            }
        }
    }
}

```

```

        toolbarUtilityButton(label: LocLabel("刷新", systemImage: "arrow.clockwise"), help: LocaleManager.loc("重新读取当前文件并刷新汇总表")) {
            viewModel.reloadFiles()
        }
        .disabled(viewModel.selectedFilePaths.isEmpty)

        toolbarUtilityButton(label: LocLabel("清空", systemImage: "xmark"), help: LocaleManager.loc("清空当前工作区，不影响原始 Excel 文件"))
    {
        viewModel.closeAllFiles()
    }
        .disabled(viewModel.selectedFilePaths.isEmpty)
    }
    .padding(4)
    .background(Color(NSColor.controlBackgroundColor))
    .overlay(
        RoundedRectangle(cornerRadius: 12, style: .continuous)
            .stroke(Color.secondary.opacity(0.12), lineWidth: 1)
    )
    .clipShape(RoundedRectangle(cornerRadius: 12, style: .continuous))
}

Spacer()

if hasWorkspace {
    toolbarPrimaryButton(
        label: LocLabel("导出 XLSX", systemImage: "square.and.arrow.up"),
        prominent: presentation.exportIsProminent,
        help: viewModel.canExport ? LocaleManager.loc("导出同构汇总 Excel") : LocaleManager.loc("当前没有可导出的汇总表结果")
    ) {
        viewModel.exportResult()
    }
    .disabled(!viewModel.canExport)
}

}
.padding(.horizontal)
.padding(.vertical, 10)
.background(Color(NSColor.windowBackgroundColor))
}

private var toolbarStripDivider: some View {
    Rectangle()
        .fill(Color.secondary.opacity(0.10))
        .frame(width: 1, height: 18)
        .padding(.horizontal, 2)
}

private func toolbarPrimaryButton(
    label: some View,
    prominent: Bool,
    help: String,
    action: @escaping () -> Void
) -> some View {
    Button(action: action) {
        label
    }
    .buttonStyle(WorkspaceChromePrimaryButtonStyle(prominent: prominent))
    .help(help)
}

private func toolbarUtilityButton(
    label: some View,
    help: String,

```



```

        action: @escaping () -> Void
    ) -> some View {
        Button(action: action) {
            label
        }
        .buttonStyle(WorkspaceChromeUtilityButtonStyle())
        .help(help)
    }

private var emptyView: some View {
    ZStack {
        LinearGradient(
            colors: [
                Color(NSColor.controlBackgroundColor),
                Color.white.opacity(0.92)
            ],
            startPoint: .top,
            endPoint: .bottom
        )

        RoundedRectangle(cornerRadius: 28, style: .continuous)
            .fill(Color.white.opacity(isDropTargeted ? 0.94 : 0.72))
            .frame(maxWidth: 560, maxHeight: 360)
            .overlay {
                EmptyWorkspaceBackdrop()
                .clipShape(RoundedRectangle(cornerRadius: 28, style: .continuous))
            }
            .overlay {
                RoundedRectangle(cornerRadius: 28, style: .continuous)
                .strokeBorder(dropZoneBorderColor, lineWidth: isDropTargeted ? 2 : 1)
            }
            .shadow(color: Color.black.opacity(isDropTargeted ? 0.08 : 0.04), radius: 24, y: 12)
            .overlay {
                VStack(spacing: 18) {
                    EmptyWorkspaceArtwork(isHighlighted: isDropTargeted)

                    VStack(spacing: 8) {
                        Text(isDropTargeted ? LocaleManager.loc("松手即可导入") : LocaleManager.loc("拖入 Excel 文件"))
                            .font(.system(size: 28, weight: .semibold))
                            .foregroundColor(.primary)

                        Text(isDropTargeted ? LocaleManager.loc("支持多个 .xlsx / .xls") : LocaleManager.loc("支持多个 .xlsx / .xls"))
                            .font(.subheadline)
                            .foregroundColor(.secondary)
                    }
                    .multilineTextAlignment(.center)

                    Button {
                        viewModel.showOpenFileDialog()
                    } label: {
                        Label(LocaleManager.loc("选择文件"), systemImage: "folder.badge.plus")
                    }
                    .buttonStyle(WorkspaceChromePrimaryButtonStyle(prominent: true))
                    .controlSize(.large)
                }
                .padding(.horizontal, 40)
                .padding(.vertical, 36)
            }
            .padding(40)
    }
}

```

```

        .frame(maxWidth: .infinity, maxHeight: .infinity)
        .animation(.easeInOut(duration: 0.18), value: isDropTargeted)
    }

    private var dropZoneBorderColor: Color {
        isDropTargeted
            ? Color.accentColor.opacity(0.45)
            : Color.secondary.opacity(0.14)
    }

    private var loadingView: some View {
        VStack(spacing: 16) {
            ProgressView()
                .scaleEffect(1.4)
            Text(LocaleManager.loc("正在校验工作簿结构并准备汇总工作台..."))
                .foregroundColor(.secondary)
            Text(String(format: LocaleManager.loc("已选 %lld 个文件"), viewModel.selectedFilePaths.count))
                .font(.caption)
                .foregroundColor(.secondary)
        }
        .frame(maxWidth: .infinity, maxHeight: .infinity)
        .background(Color(NSColor.controlBackgroundColor))
    }

    private var blockedView: some View {
        ScrollView {
            VStack(alignment: .leading, spacing: 20) {
                HStack(alignment: .top, spacing: 12) {
                    Image(systemName: "exclamationmark.triangle.fill")
                        .foregroundColor(.orange)
                        .font(.title2)
                    VStack(alignment: .leading, spacing: 6) {
                        Text(LocaleManager.loc("没有可参与汇总的同构工作表"))
                            .font(.title3)
                            .fontWeight(.semibold)
                        Text(LocaleManager.loc("系统已忽略尾部空白行列后重试校验，但当前仍没有任何 sheet 能在所有文件间对齐。"))
                            .foregroundColor(.secondary)
                    }
                }
            }

            if let report = viewModel.validationReport {
                validationSummary(report)
                if report.skippedSheetCount > 0 {
                    skippedSheetList(report)
                }
                fileParticipationList(report.files)
            }
        }
        .padding(24)
        .frame(maxWidth: .infinity, alignment: .leading)
    }
    .background(Color(NSColor.controlBackgroundColor))
}

private func validationSummary(_ report: WorkbookValidationReport) -> some View {
    HStack(spacing: 12) {
        statCard(title: LocaleManager.loc("参与文件"), value: "\\(report.includedFiles.count)", tint: .green)
        statCard(title: LocaleManager.loc("阻断文件"), value: "\\(report.blockedFiles.count)", tint: .red)
        statCard(title: LocaleManager.loc("警告文件"), value: "\\(report.warningFiles.count)", tint: .orange)
        if report.skippedSheetCount > 0 {

```

```

        statCard(title: LocaleManager.loc("跳过工作表"), value: "\\(report.skippedSheetCount)", tint: .orange)
    }
}
}

```

```

private fun statCard(title: String, value: String, tint: Color) -> some View {
    VStack(alignment: .Leading, spacing: 6) {
        Text(title)
            .font(.caption)
            .foregroundColor(.secondary)
        Text(value)
            .font(.title3)
            .fontWeight(.semibold)
            .foregroundColor(tint)
    }
    .padding()
    .frame(maxWidth: 140, alignment: .leading)
    .background(tint.opacity(0.1))
    .clipShape(RoundedRectangle(cornerRadius: 12))
}

```

```

private fun fileParticipationList(_ reports: [FileValidationReport]) -> some View {
    VStack(alignment: .Leading, spacing: 12) {
        Text(LocaleManager.loc("文件参与情况"))
            .font(.headline)

        ForEach(reports, id: \.filepath) { report in
            VStack(alignment: .leading, spacing: 10) {
                HStack(spacing: 10) {
                    statusDot(for: report.status)
                    Text(report.filename)
                        .fontWeight(.medium)
                    Spacer()
                    Text(report.statusLabel)
                        .font(.caption)
                        .foregroundColor(report.statusColor)
                }

                if !report.issues.isEmpty {
                    VStack(alignment: .Leading, spacing: 6) {
                        ForEach(report.issues) { issue in
                            Text(issue.message)
                                .font(.caption)
                                .foregroundColor(issue.severity == .blocking ? .red : .orange)
                        }
                    }
                }
            }
        }
    }
    .padding()
    .background(Color.white)
    .clipShape(RoundedRectangle(cornerRadius: 12))
    .overlay(
        RoundedRectangle(cornerRadius: 12)
            .stroke(report.statusColor.opacity(0.2), lineWidth: 1)
    )
}
}
}

```

```

private fun skippedSheetList(_ report: WorkbookValidationReport) -> some View {

```